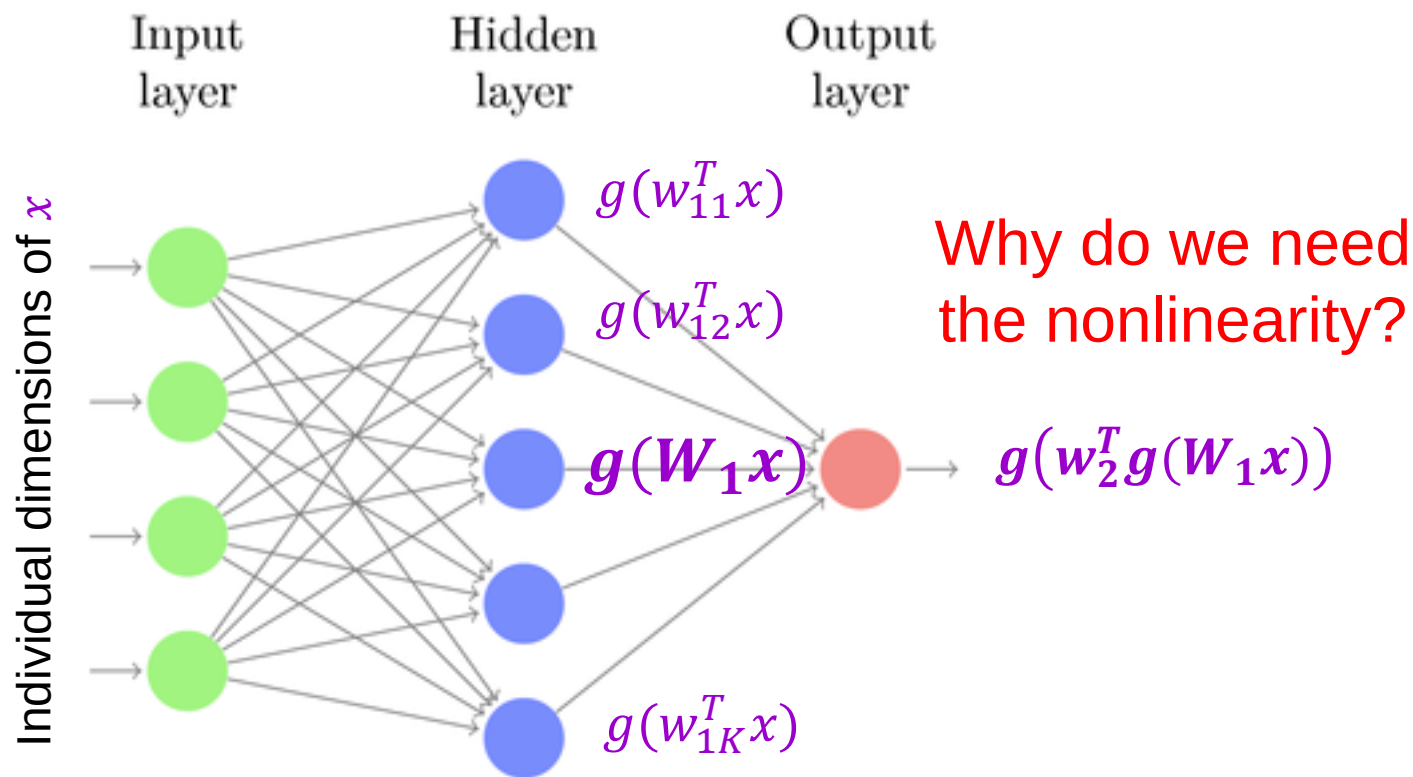


Multi-layer neural networks and backpropagation

Jianguo Zhang

Two-layer neural network

- Introduce a *hidden layer* of perceptrons computing linear combinations of inputs followed by a *nonlinearity*

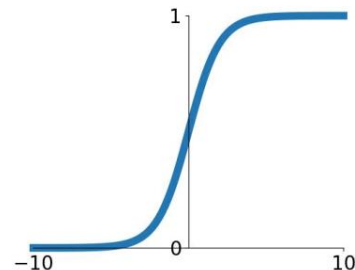


W_1 – matrix with rows w_{1j}^T

Common nonlinearities

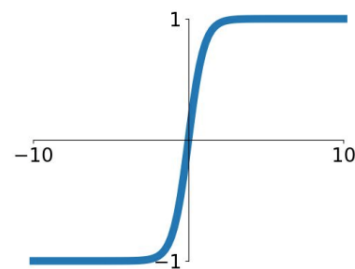
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



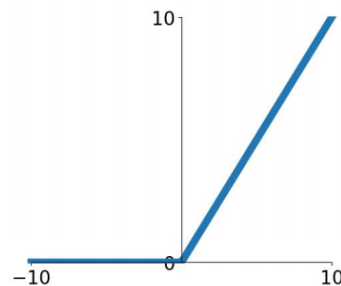
tanh

$$\tanh(x)$$



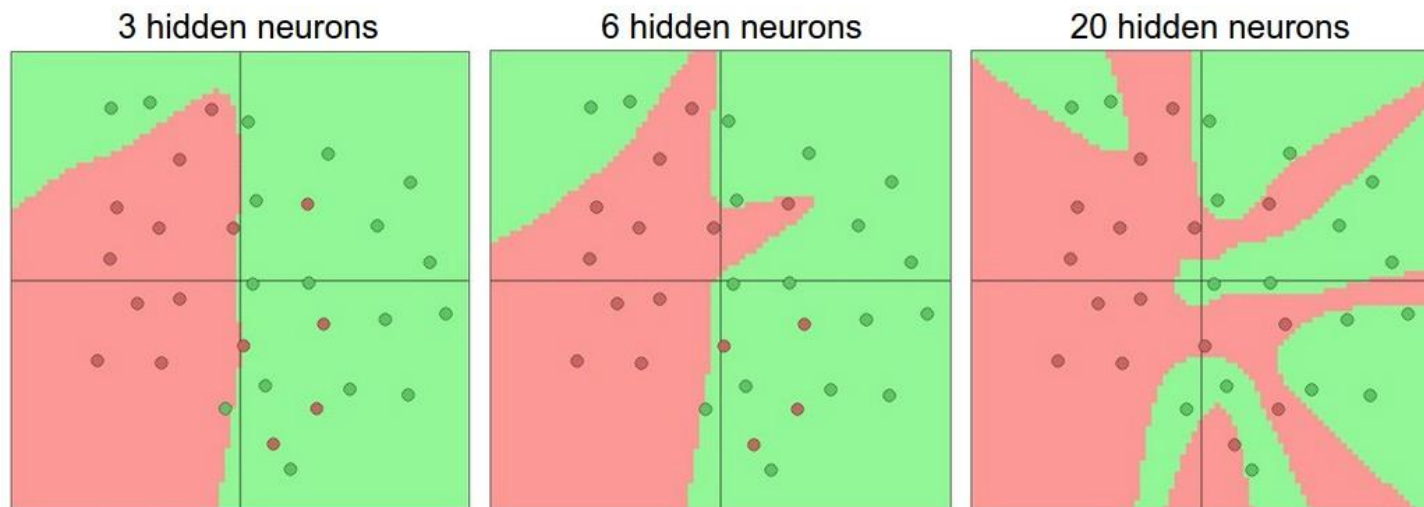
ReLU

$$\max(0, x)$$

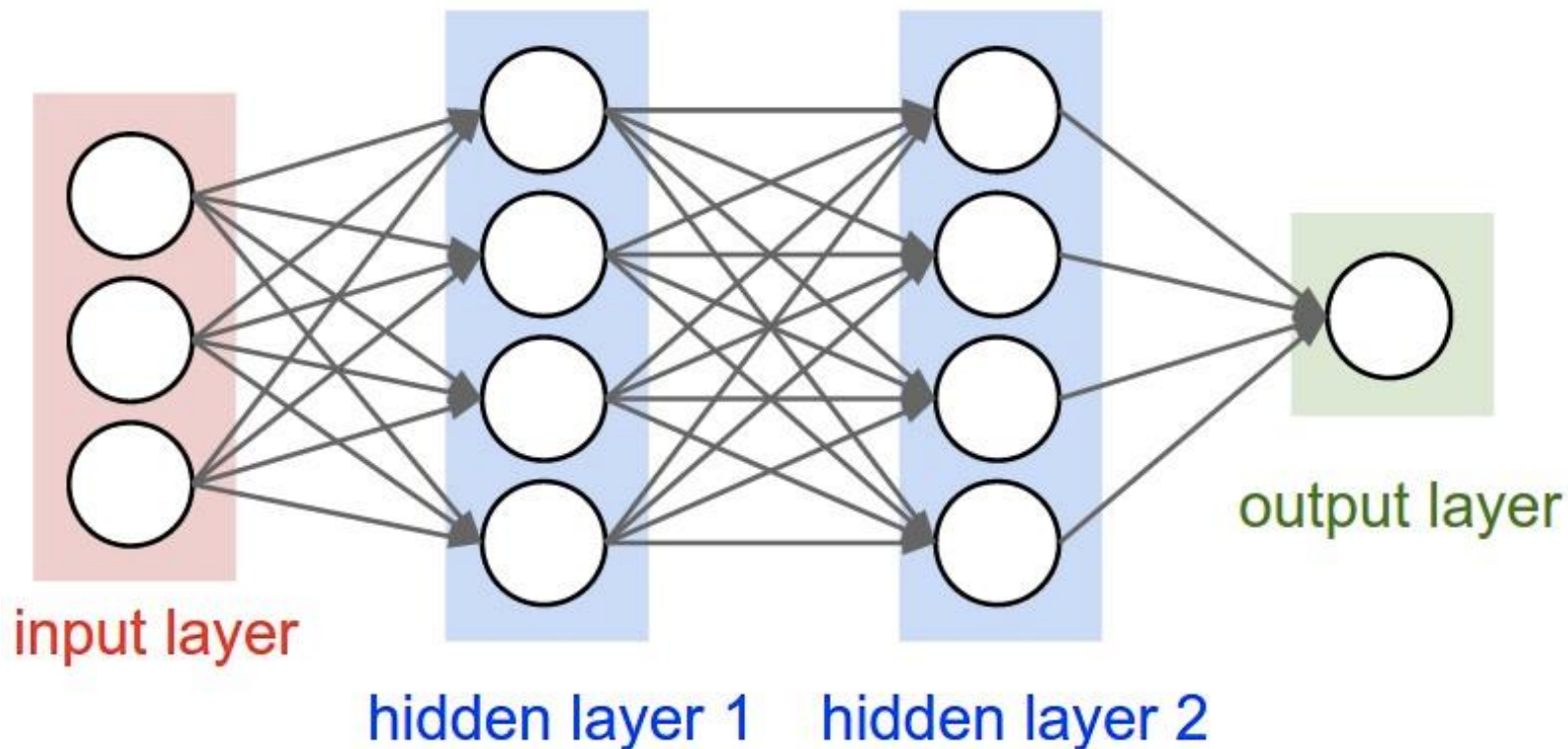


Two-layer neural network

- Introduce a *hidden layer* of perceptrons computing linear combinations of inputs followed by a *nonlinearity*
- This gives a [universal function approximator](#)
 - But the hidden layer may need to be huge



Beyond two layers



“Deep” pipeline



- Learn a *feature hierarchy*
- Each layer extracts features from the output of previous layer
- All layers are trained jointly

Multi-Layer network demo

INPUT

Which properties do you want to feed in?

+

-

1 HIDDEN LAYER

+

-

4 neurons

X_1

X_2

X_1^2

X_2^2

$X_1 X_2$

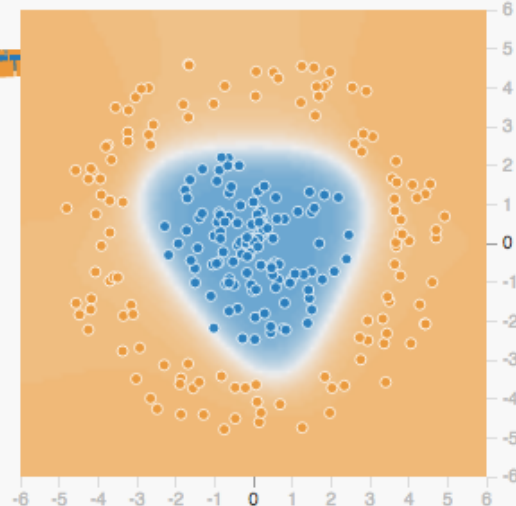
$\sin(X_1)$

$\sin(X_2)$

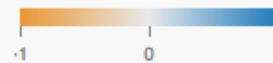
OUTPUT

Test loss 0.020

Training loss 0.013



Colors shows data, neuron and weight values.

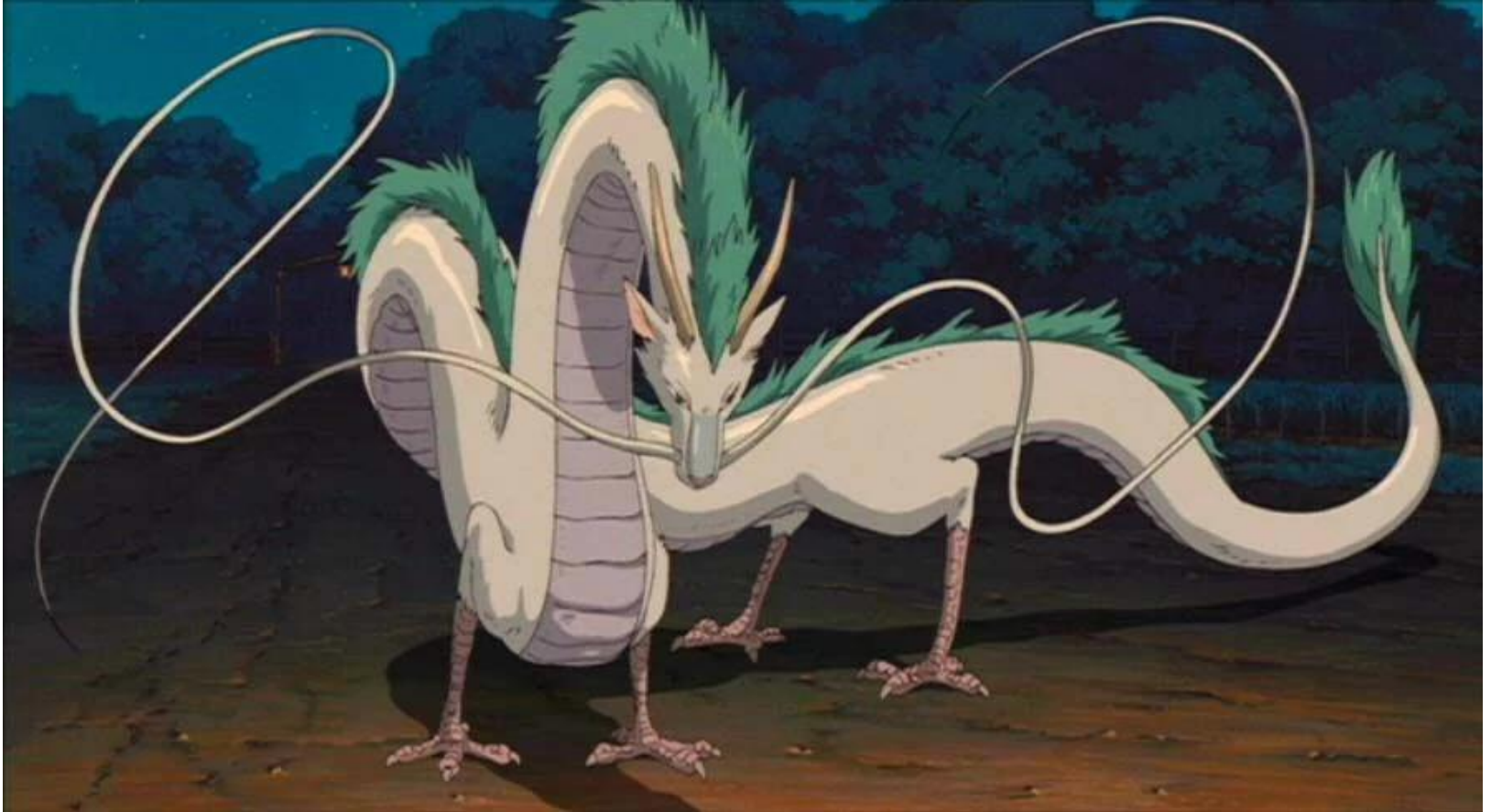


☐ Show test data

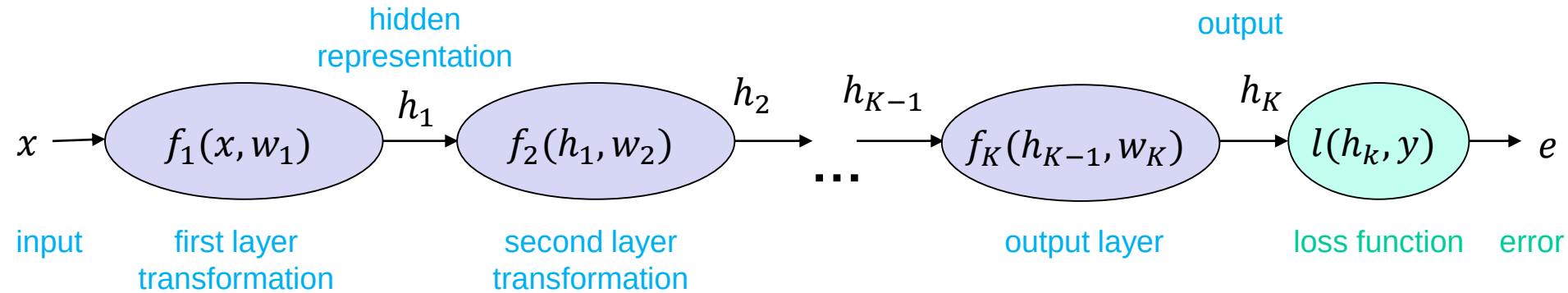
☐ Discretize output

<http://playground.tensorflow.org/>

How to train a multi-layer network?



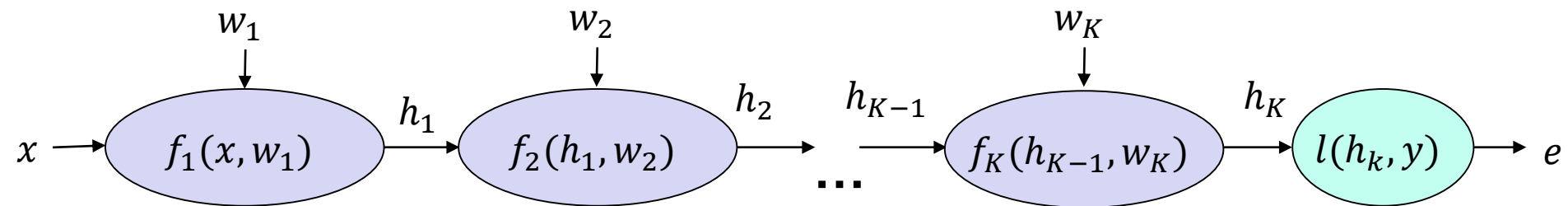
How to train a multi-layer network?



- We need to find the gradient of the error w.r.t. the parameters of each layer, $\frac{\partial e}{\partial w_k}$, to perform updates

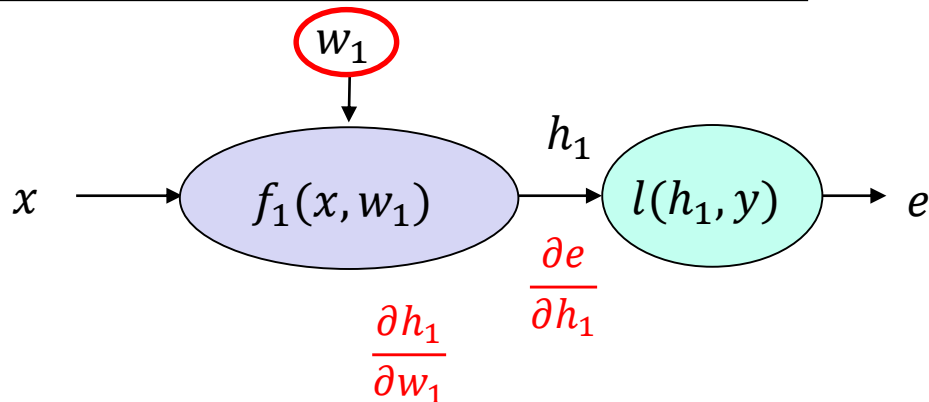
$$w_k \leftarrow w_k - \eta \frac{\partial e}{\partial w_k}$$

Computation graph



Chain rule

Let's start with $k = 1$



$$e = l(f_1(x, w_1), y)$$

$$\frac{\partial}{\partial w_1} l(f_1(x, w_1), y) =$$

Example: $e = (y - w_1^T x)^2$

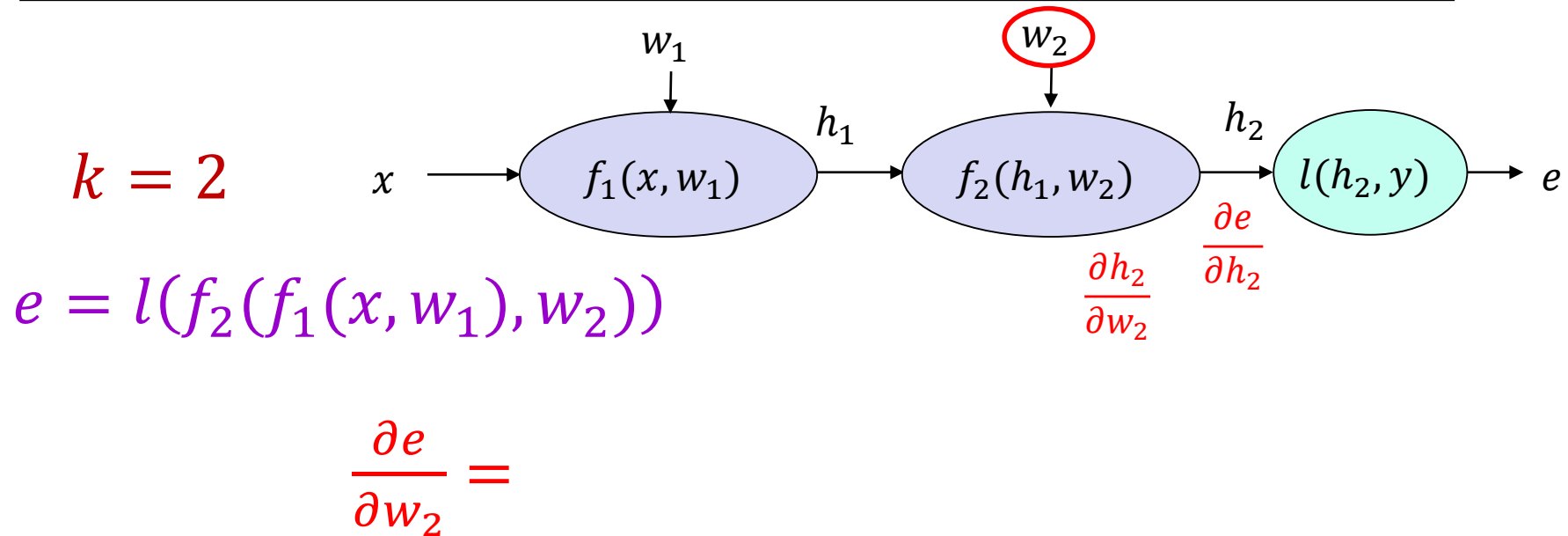
$$h_1 = f_1(x, w_1) = w_1^T x$$

$$e = l(h_1, y) = (y - h_1)^2$$

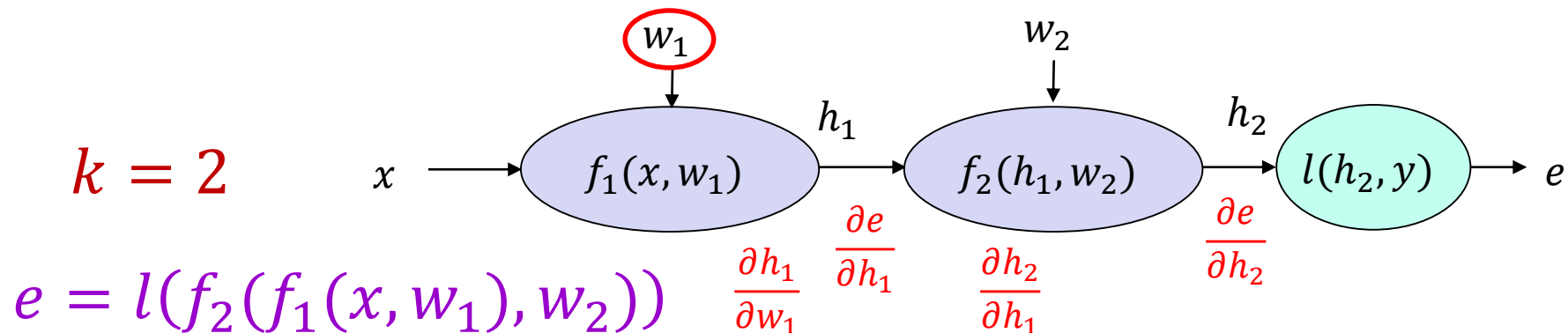
$$\frac{\partial h_1}{\partial w_1} =$$
$$\frac{\partial e}{\partial h_1} =$$

$$\frac{\partial e}{\partial w_1} = \frac{\partial e}{\partial h_1} \frac{\partial h_1}{\partial w_1}$$

Chain rule



Chain rule



$$\frac{\partial e}{\partial w_2} = \frac{\partial e}{\partial h_2} \frac{\partial h_2}{\partial w_2}$$



Example: $e = -\log(\sigma(w_1^T x))$ (assume $y = 1$)

$$h_1 = f_1(x, w_1) = w_1^T x$$

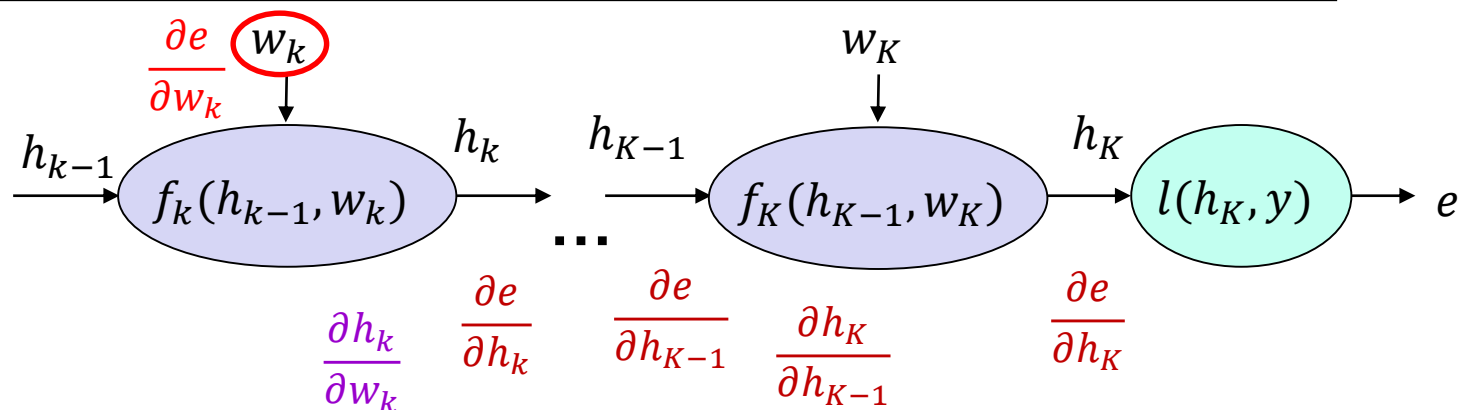
$$h_2 = f_2(h_1) = \sigma(h_1)$$

$$e = l(h_2, 1) = -\log(h_2)$$

$$\begin{aligned} \frac{\partial h_1}{\partial w_1} &= \\ \frac{\partial h_2}{\partial h_1} &= \\ \frac{\partial e}{\partial h_2} &= \end{aligned}$$

$$\frac{\partial e}{\partial w_1} = \frac{\partial e}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_1} =$$

Chain rule



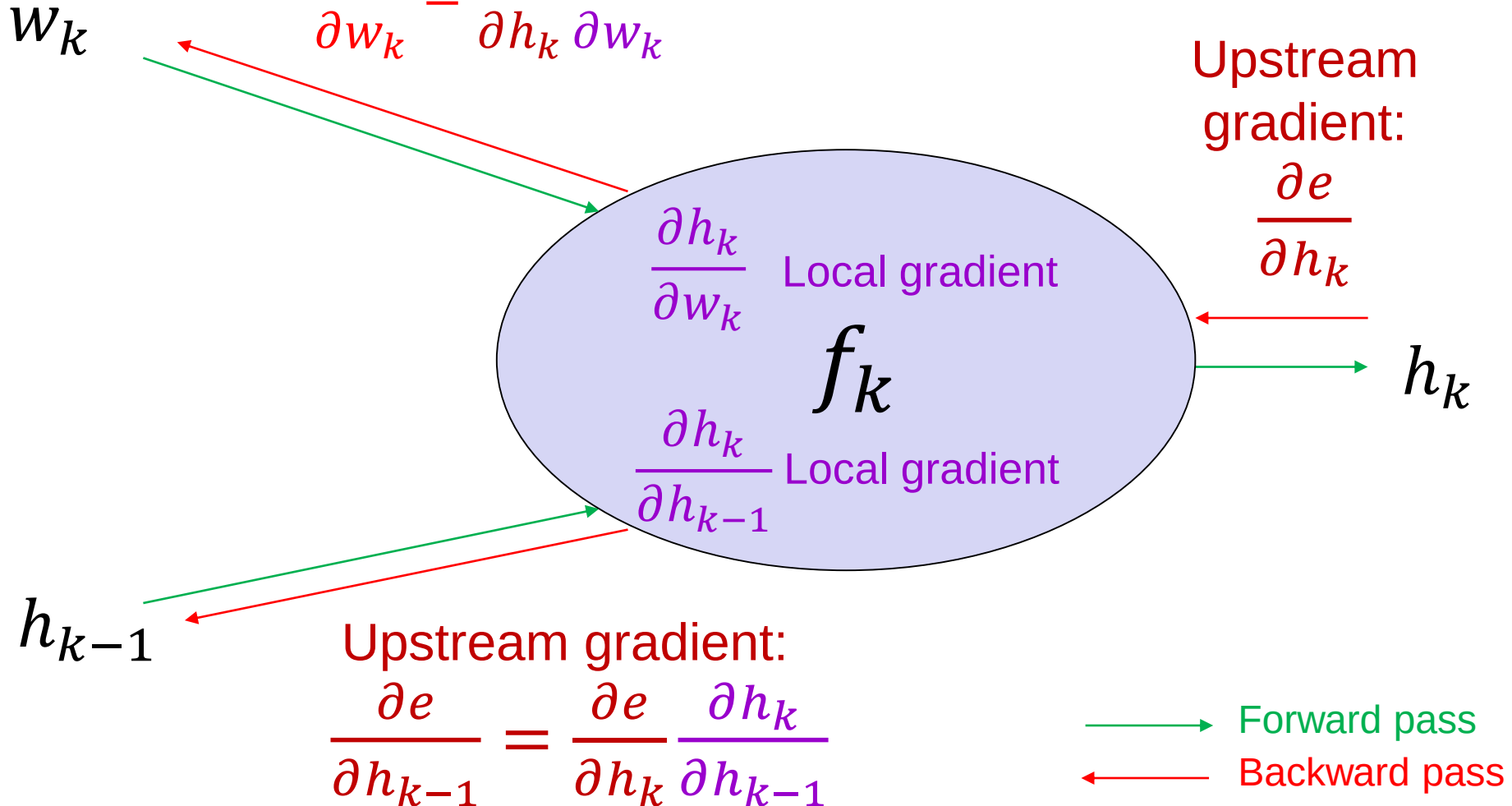
General case:

$$\frac{\partial e}{\partial w_k} = \underbrace{\frac{\partial e}{\partial h_K} \frac{\partial h_K}{\partial h_{K-1}} \cdots \frac{\partial h_{k+1}}{\partial h_k}}_{\text{Upstream gradient } \frac{\partial e}{\partial h_k}} \underbrace{\frac{\partial h_k}{\partial w_k}}_{\text{Local gradient}}$$

Backpropagation summary

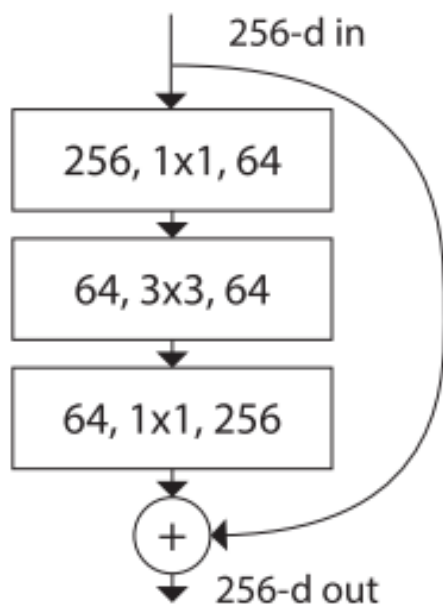
Parameter update:

$$\frac{\partial e}{\partial w_k} = \frac{\partial e}{\partial h_k} \frac{\partial h_k}{\partial w_k}$$

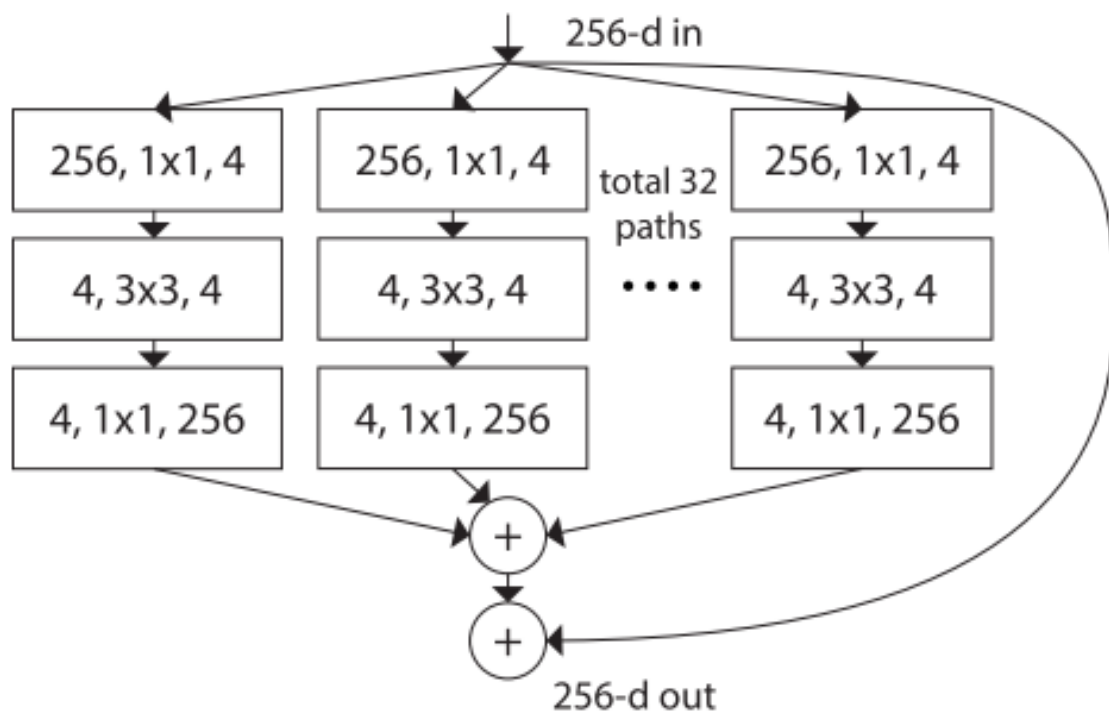


What about more general computation graphs?

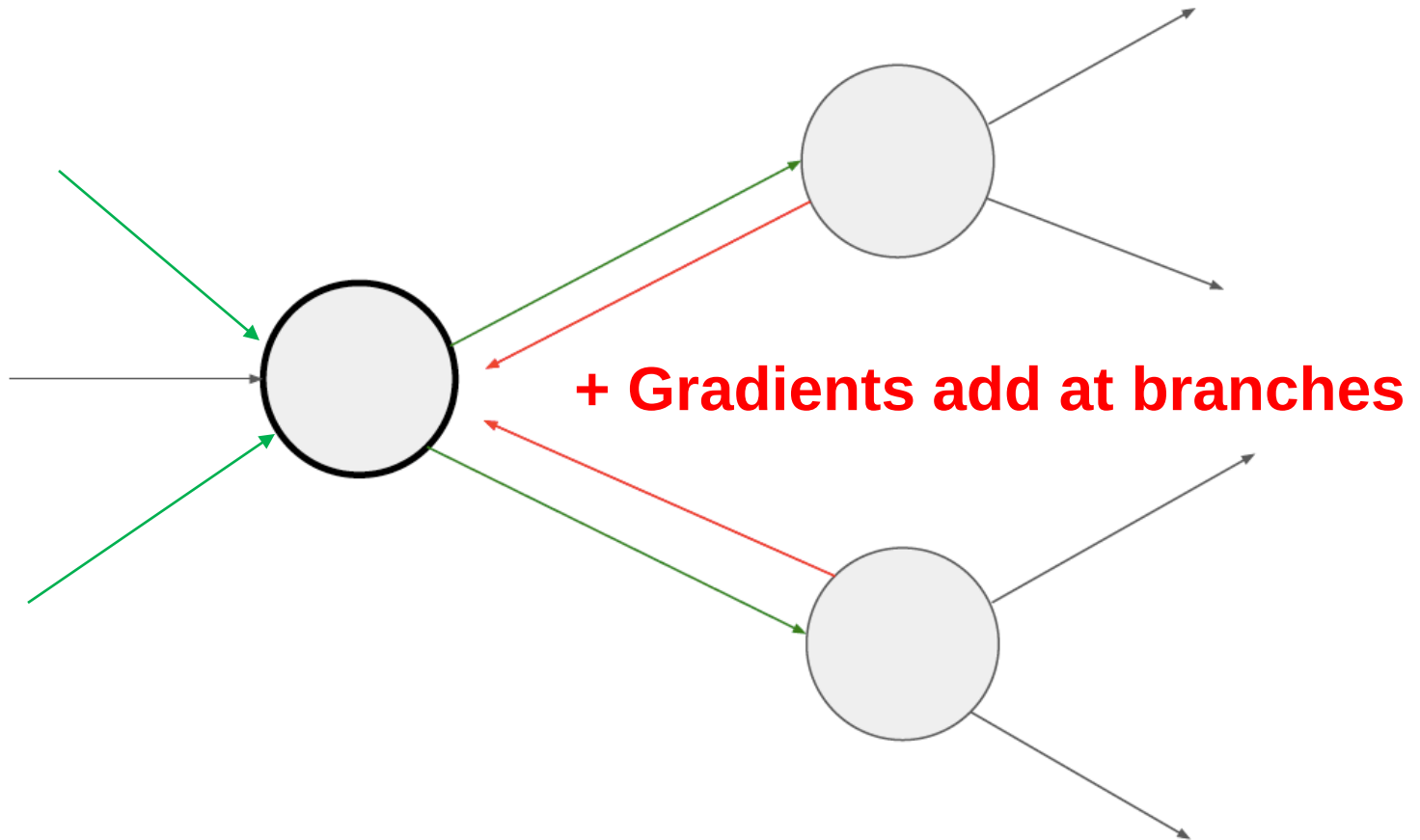
ResNet



ResNeXt

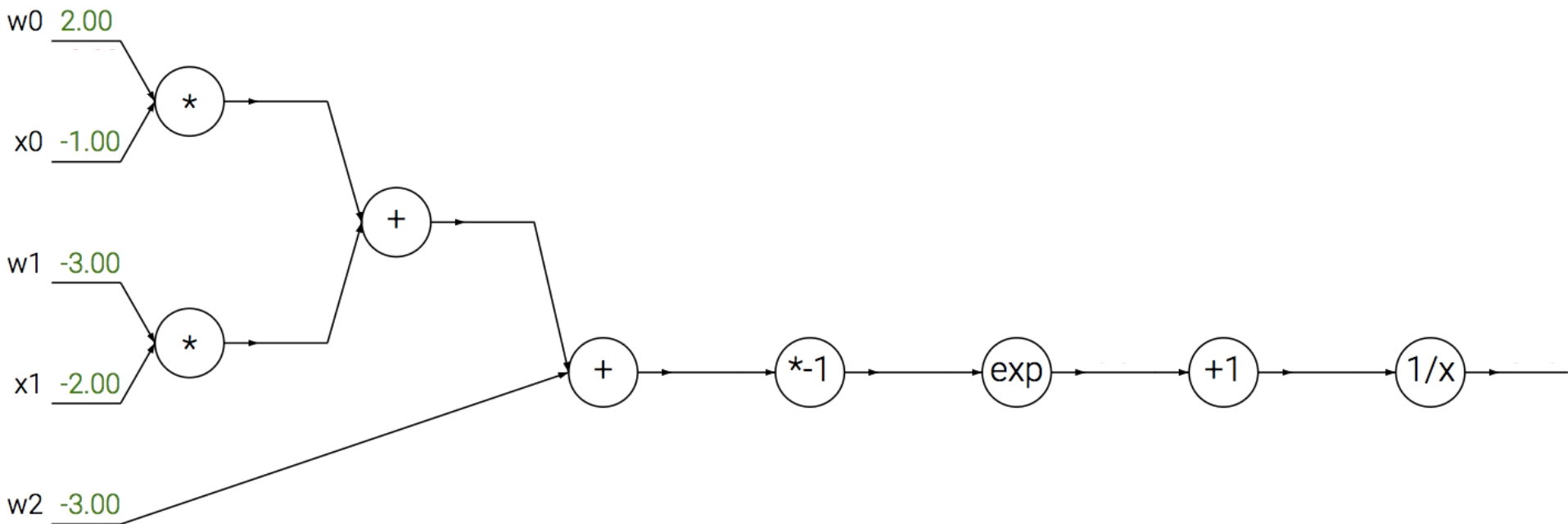


What about more general computation graphs?



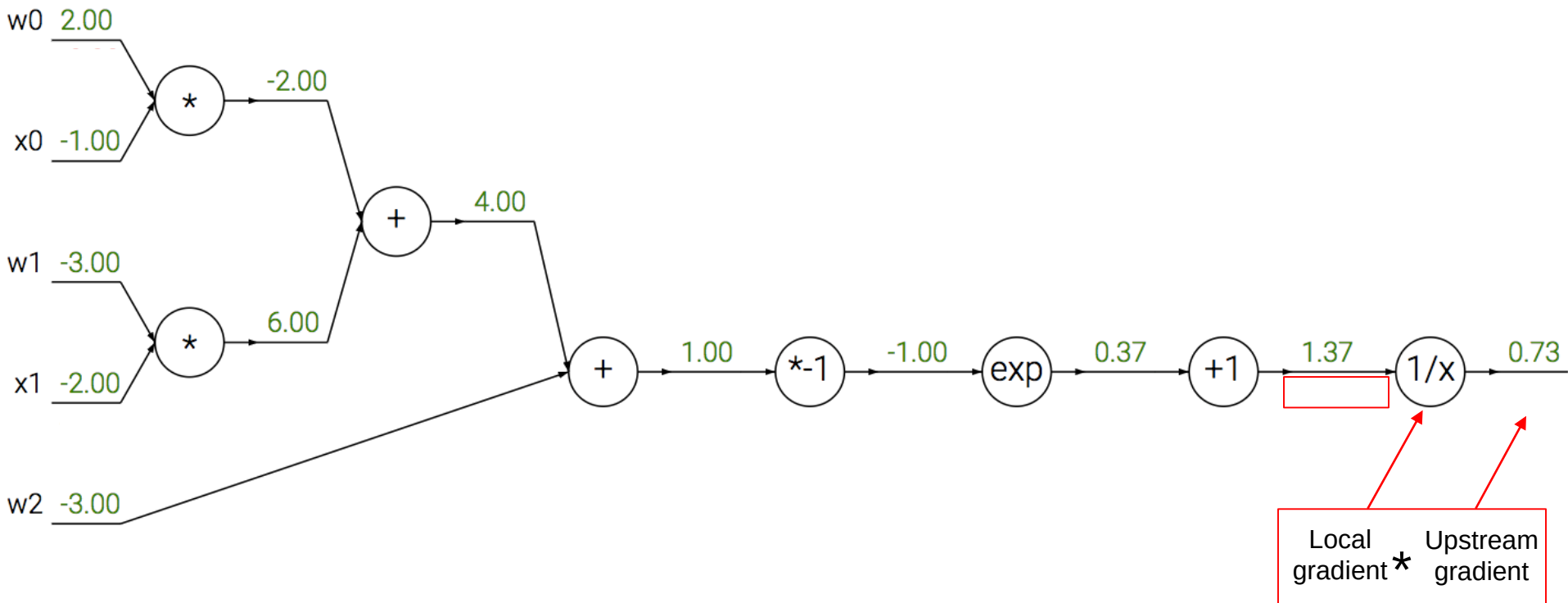
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



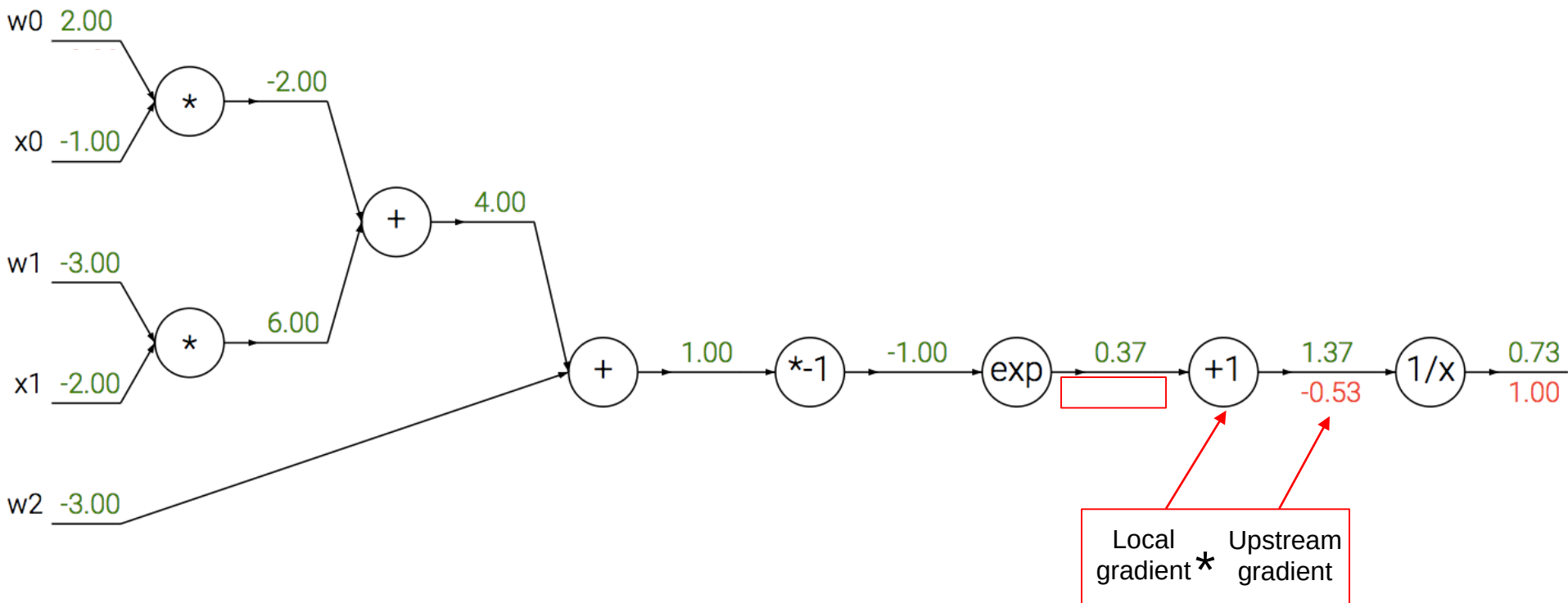
$$(1/x)' = -1/x^2$$

$$-\frac{1}{1.37^2} * 1 = -0.53$$

Source: [Stanford 231n](#)

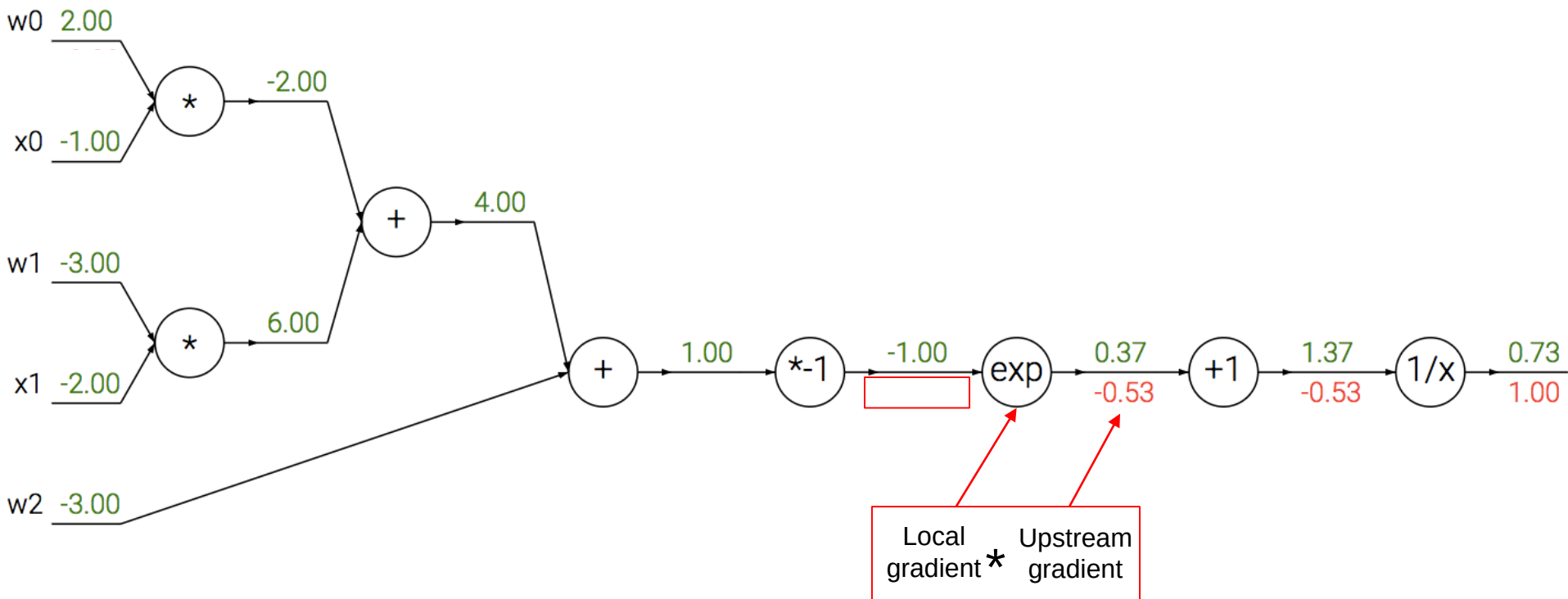
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



A detailed example

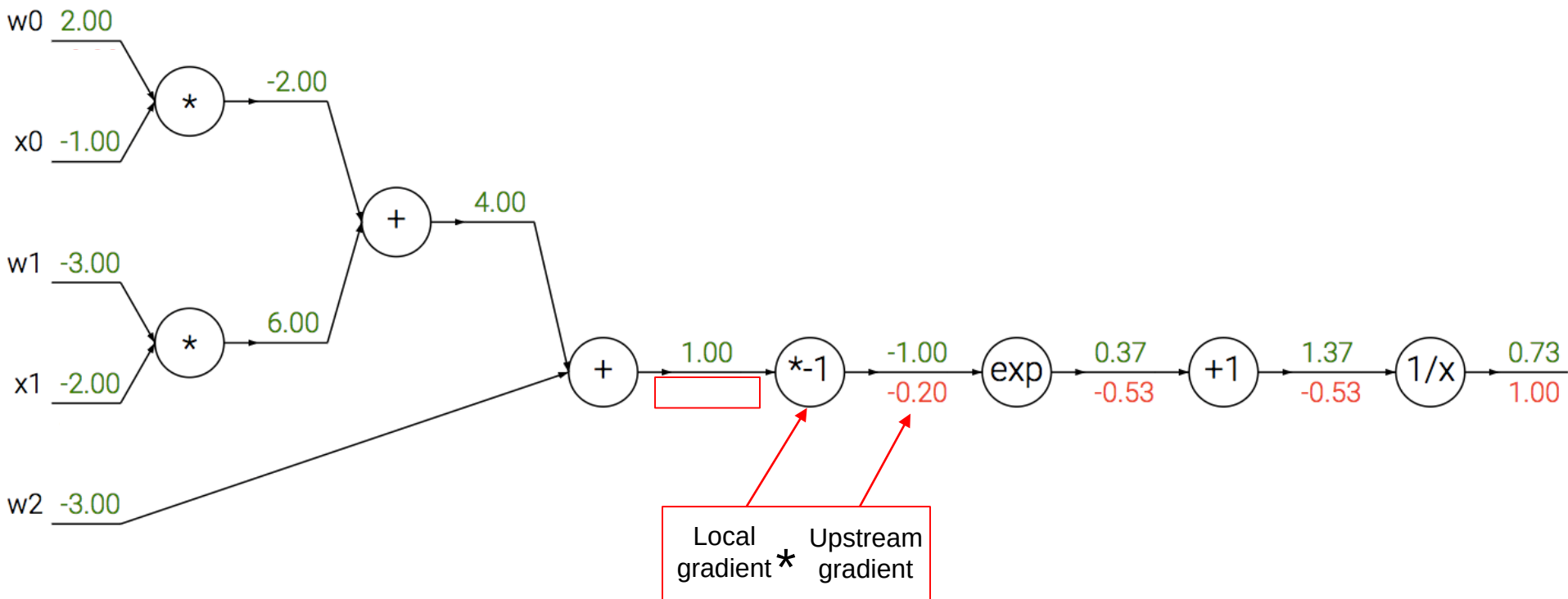
$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



$$\exp(-1) * (-0.53) = -0.20$$

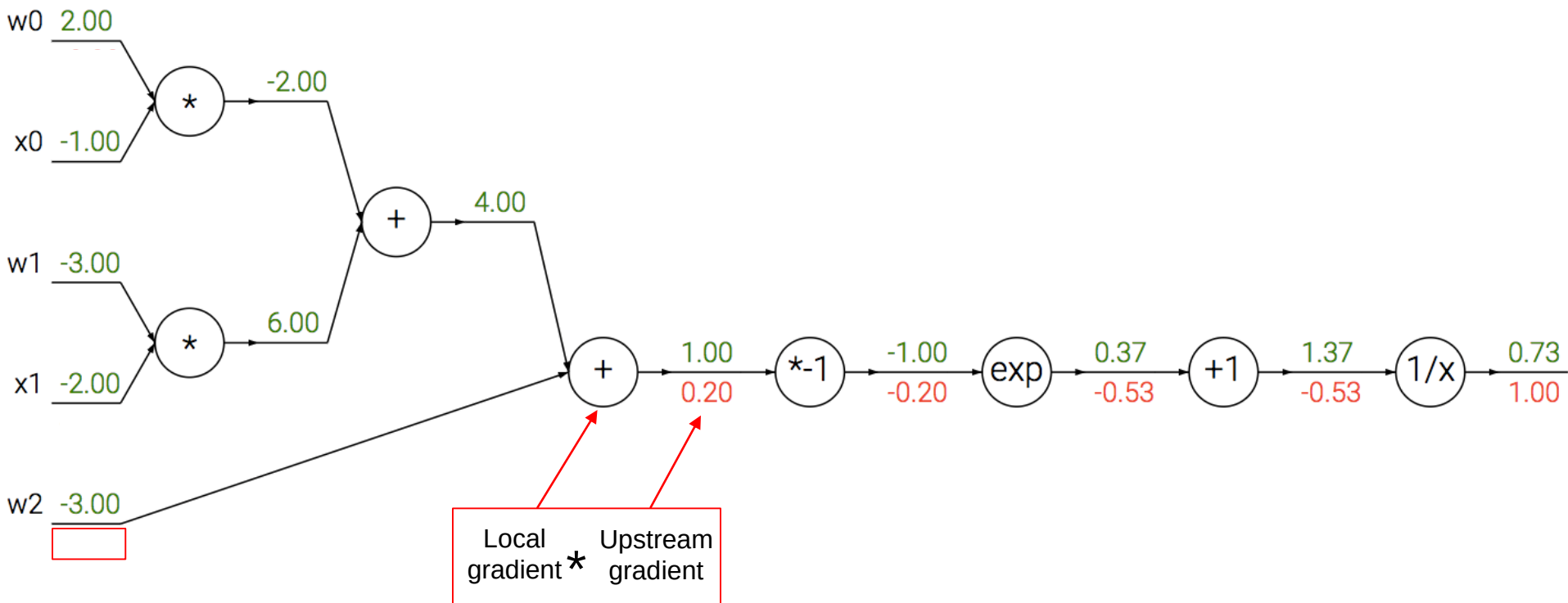
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



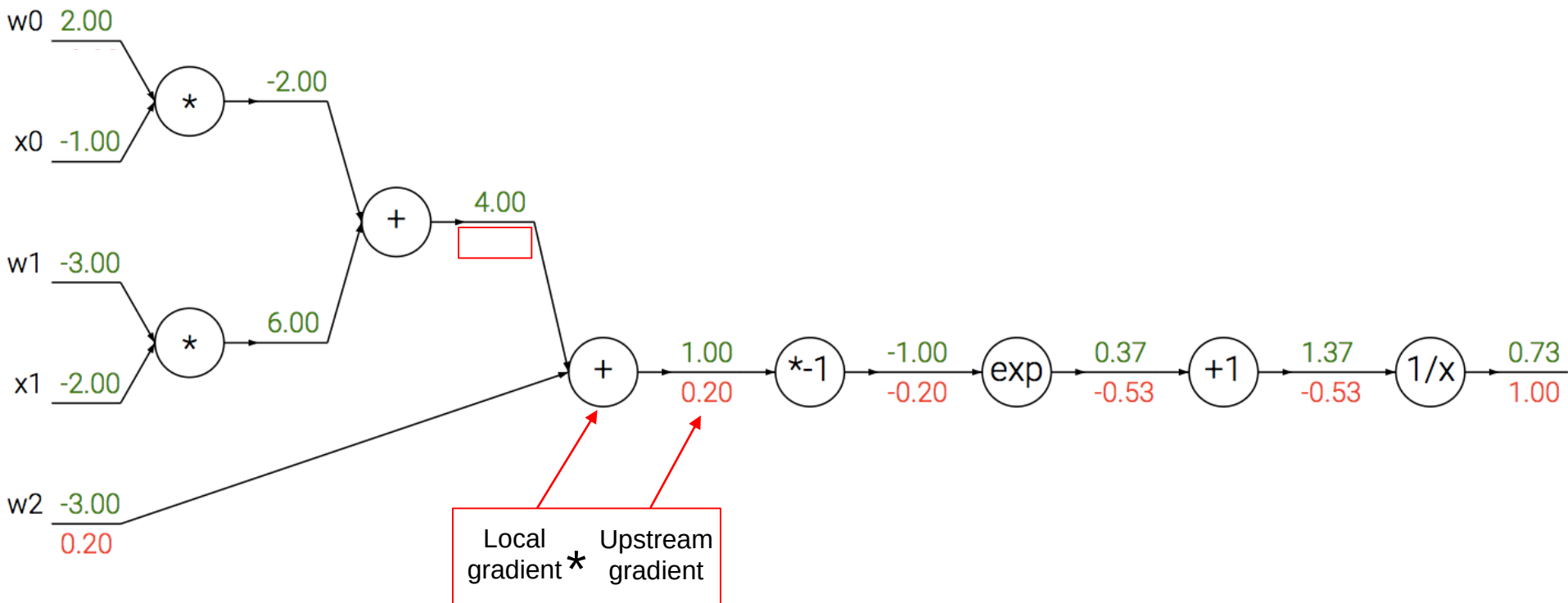
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



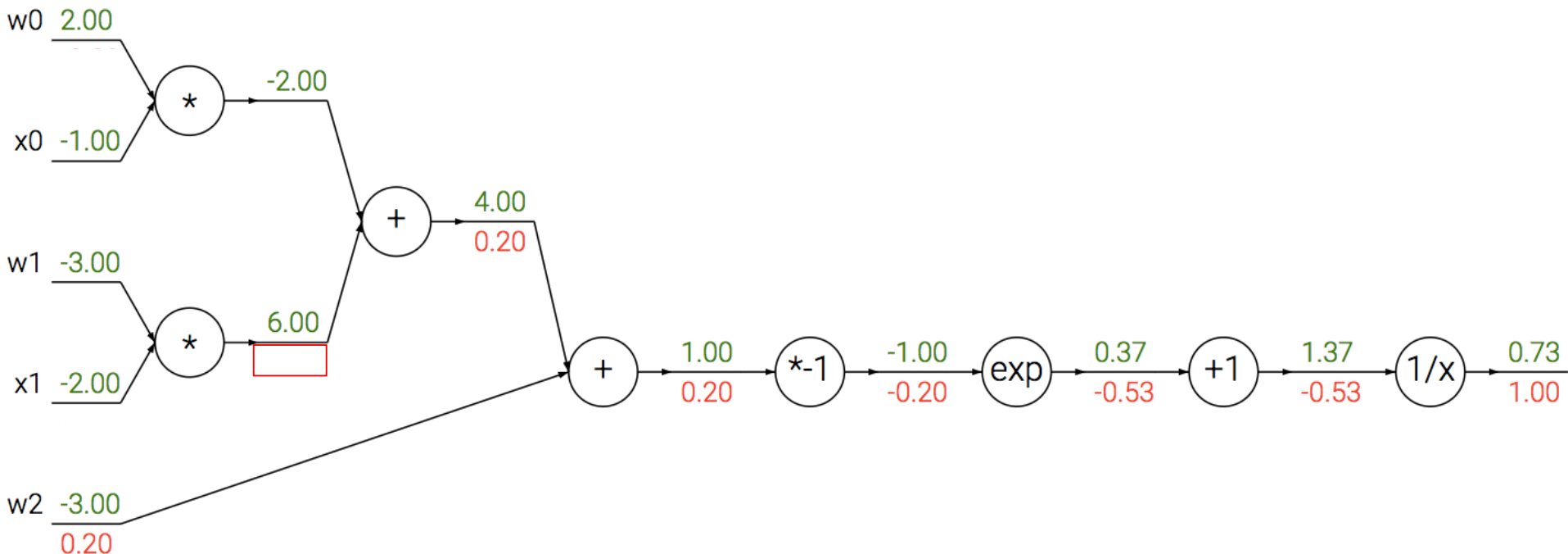
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



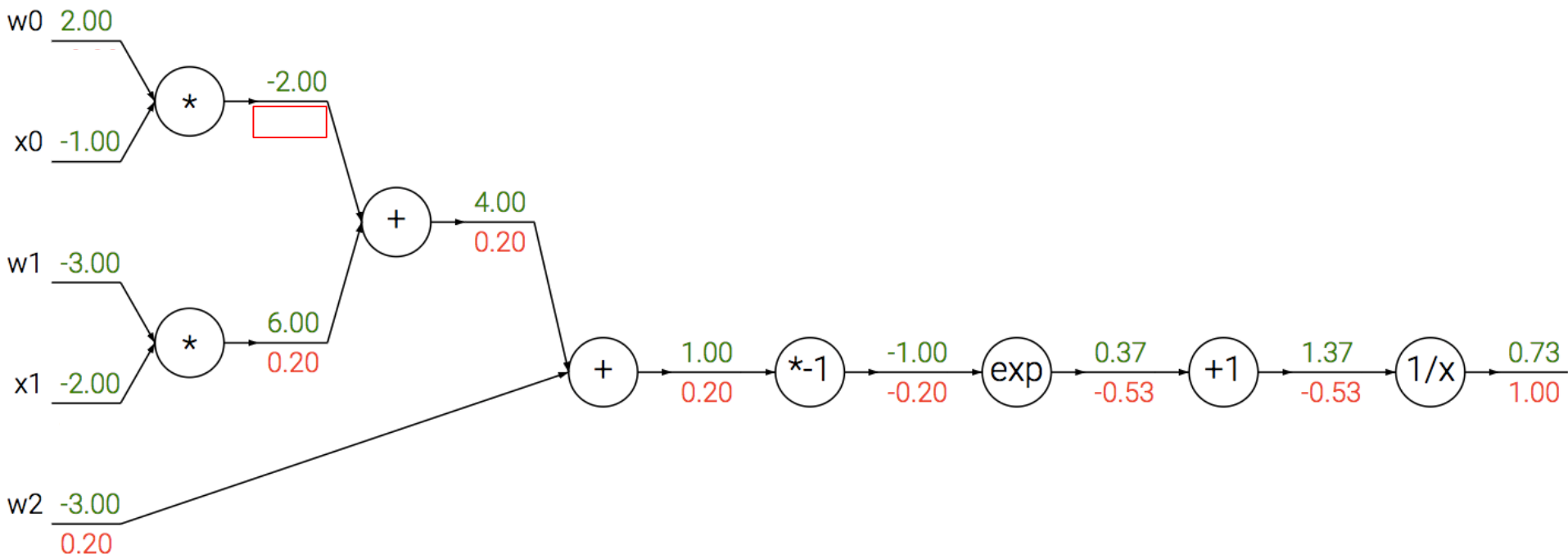
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



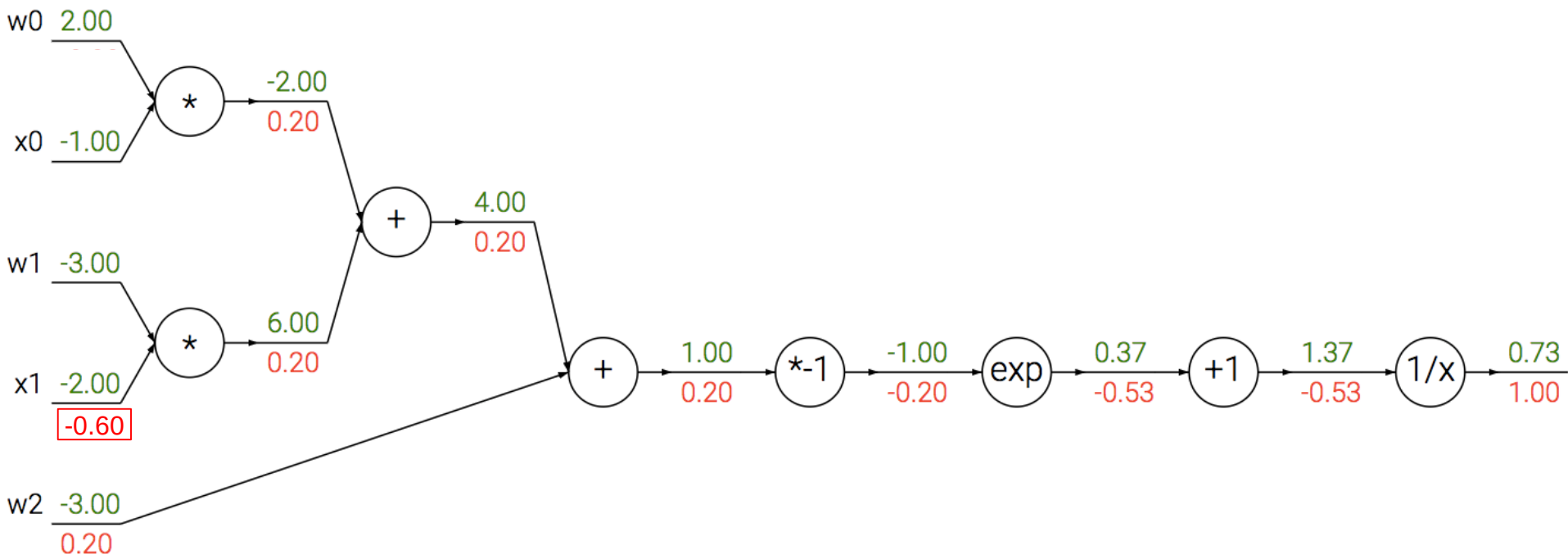
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



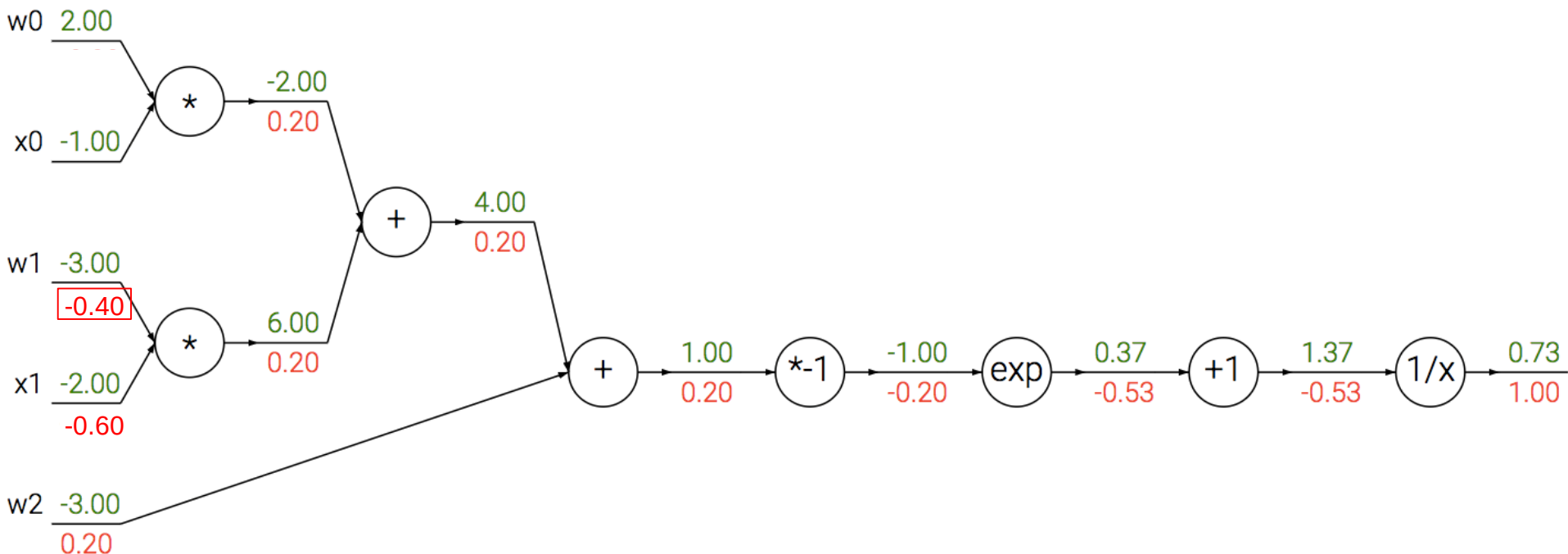
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



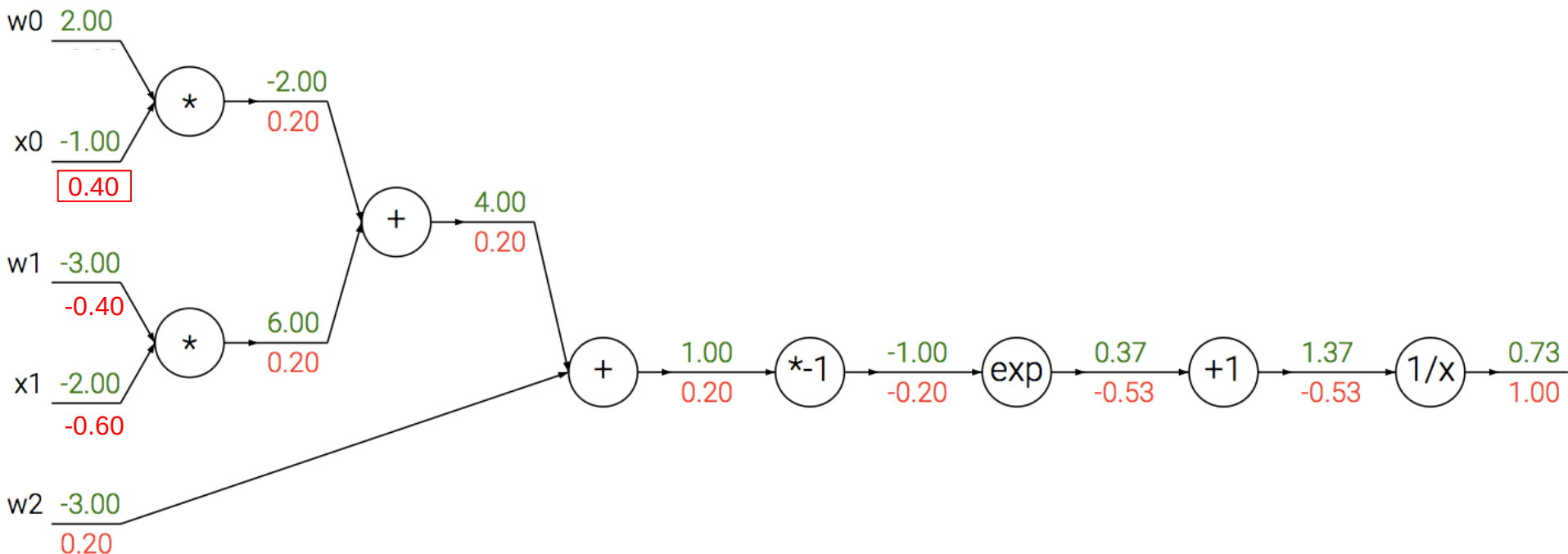
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



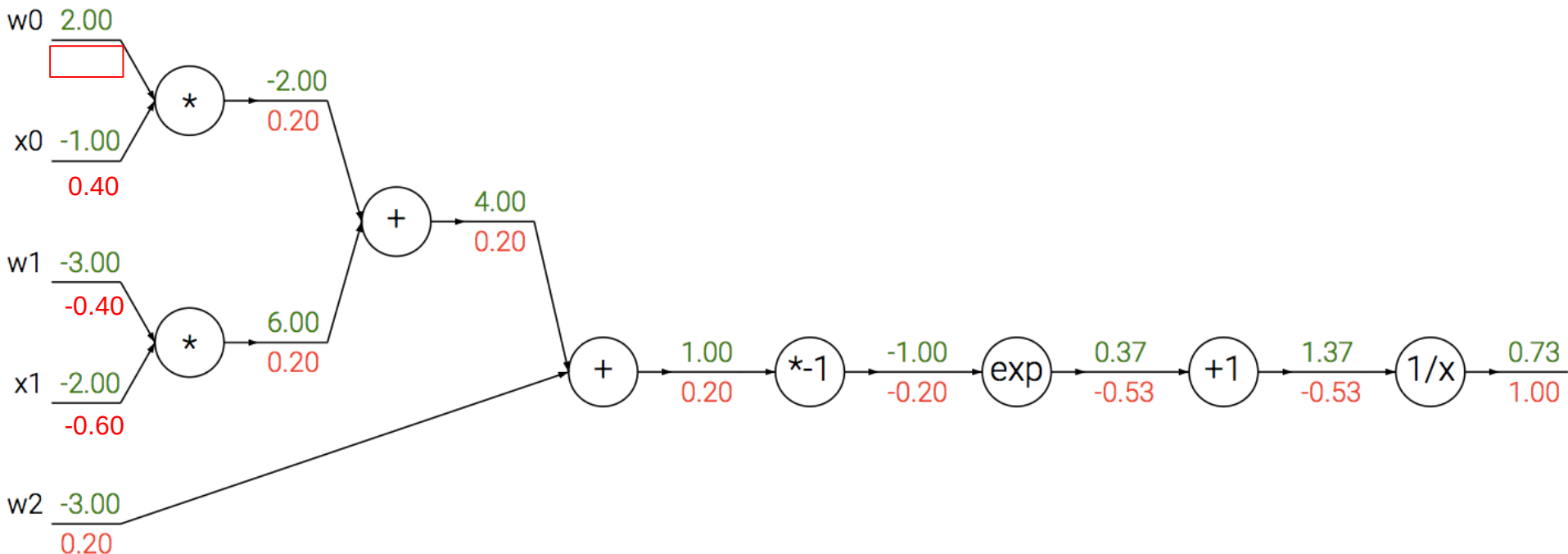
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



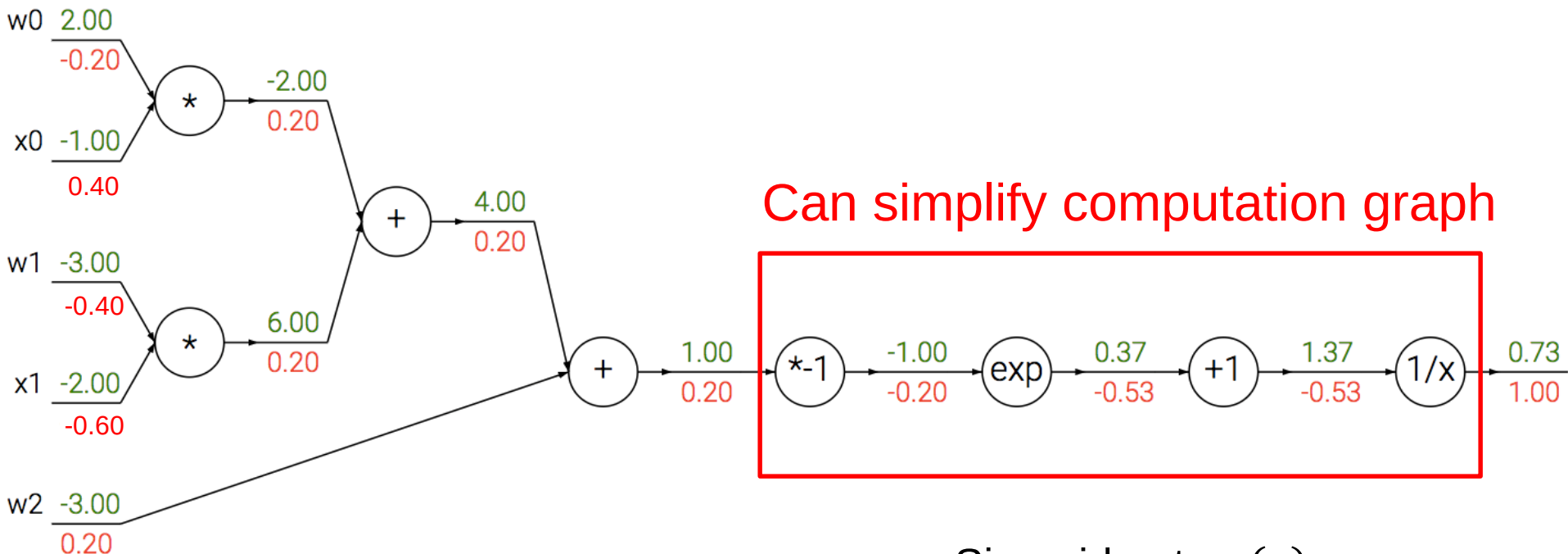
A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$



A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w_0x_0 + w_1x_1 + w_2)]}$$

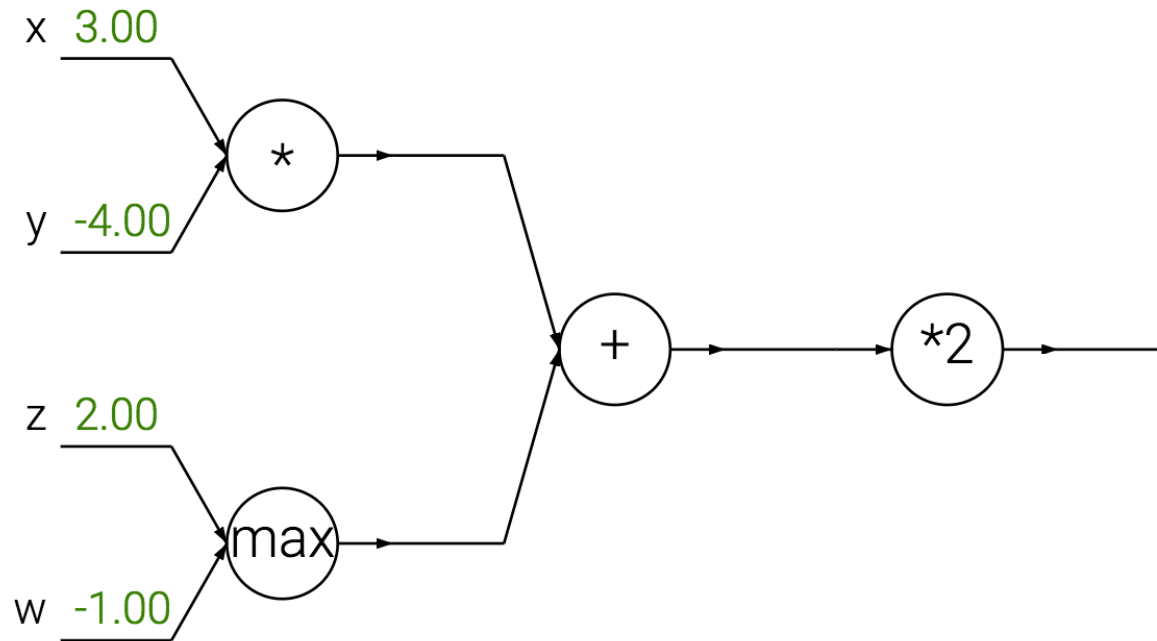


Sigmoid gate $\sigma(x)$

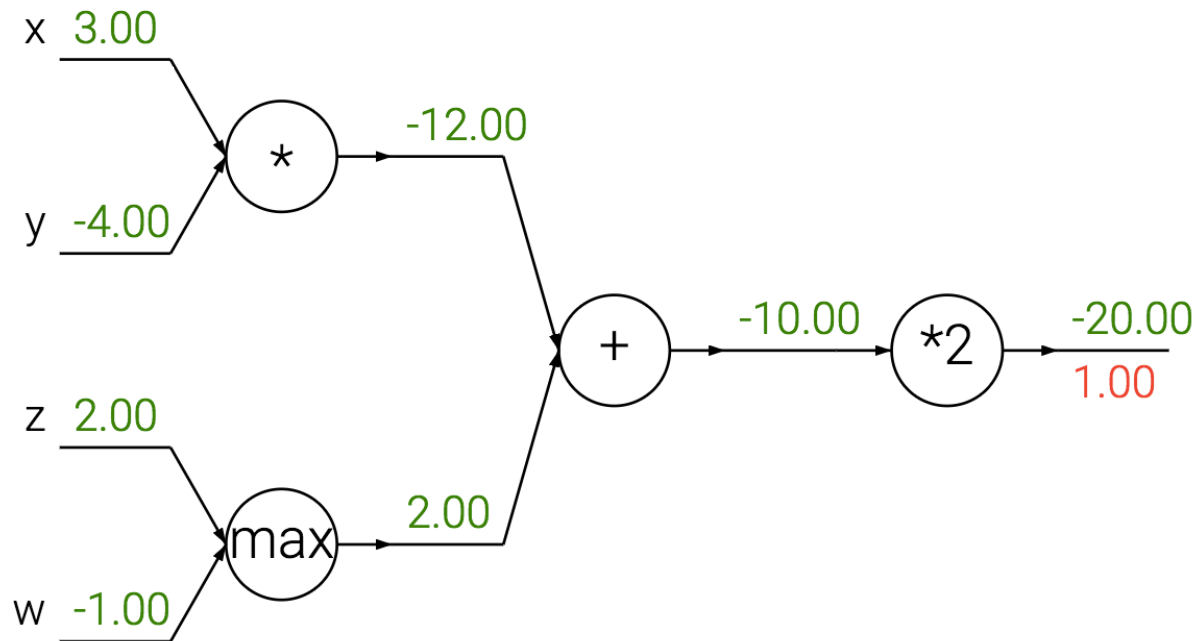
$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

$$\sigma(1)(1 - \sigma(1)) = 0.73 * (1 - 0.73) = 0.20$$

Patterns in gradient flow

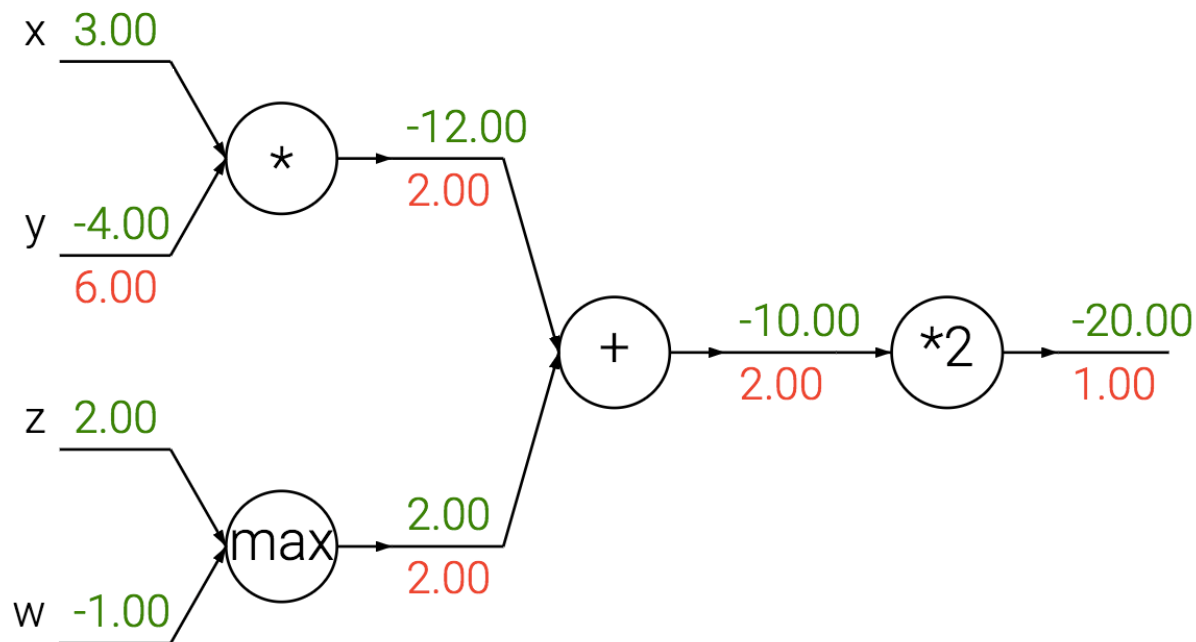


Patterns in gradient flow



Add gate: “gradient distributor”

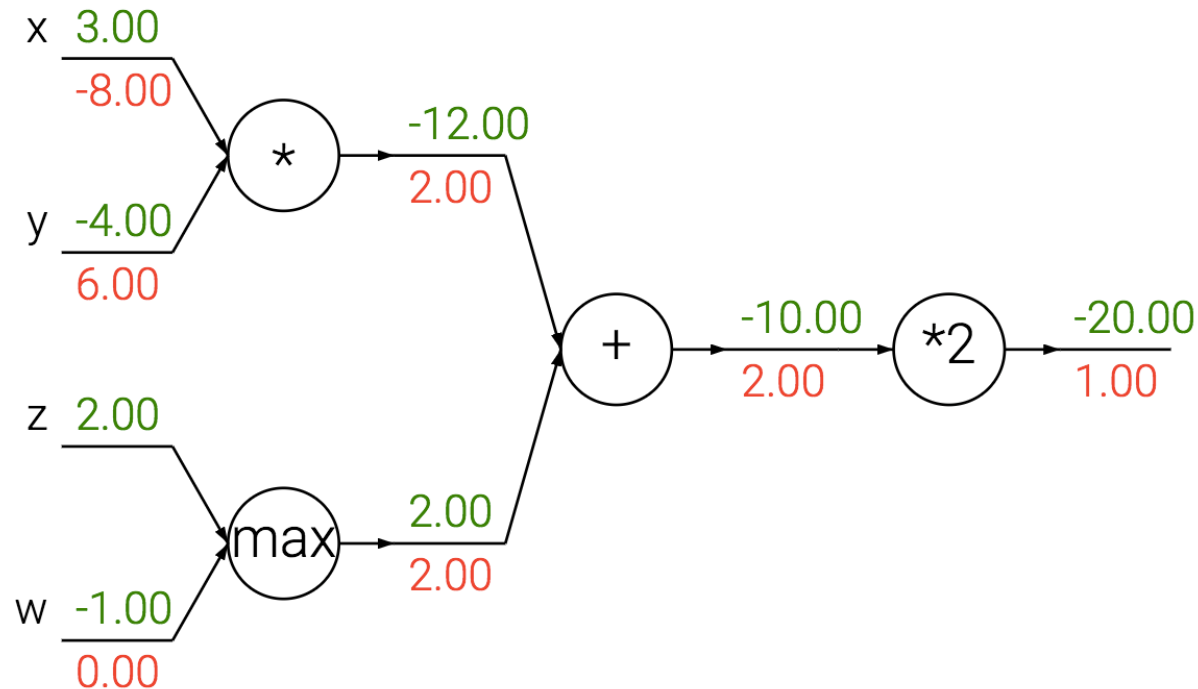
Patterns in gradient flow



Add gate: “gradient distributor”

Multiply gate: “gradient switcher”

Patterns in gradient flow



Add gate: “gradient distributor”

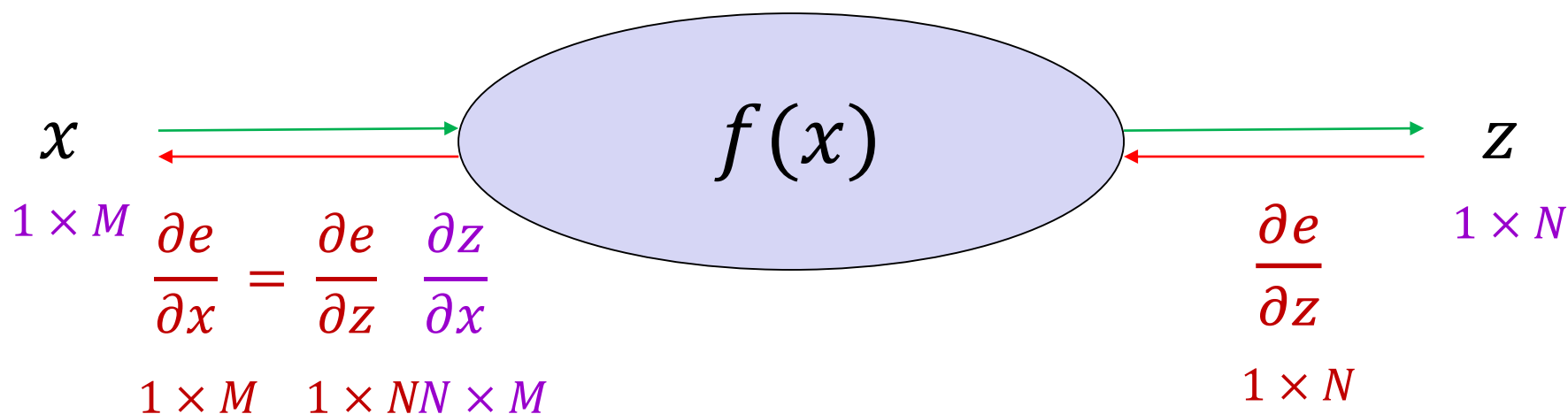
Multiply gate: “gradient switcher”

Max gate: “gradient router”

Dealing with vectors

$$\frac{\partial z}{\partial x} = \begin{pmatrix} \frac{\partial z_1}{\partial x_1} & \cdots & \frac{\partial z_1}{\partial x_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_N}{\partial x_1} & \cdots & \frac{\partial z_N}{\partial x_M} \end{pmatrix}$$

$N \times M$
Jacobian

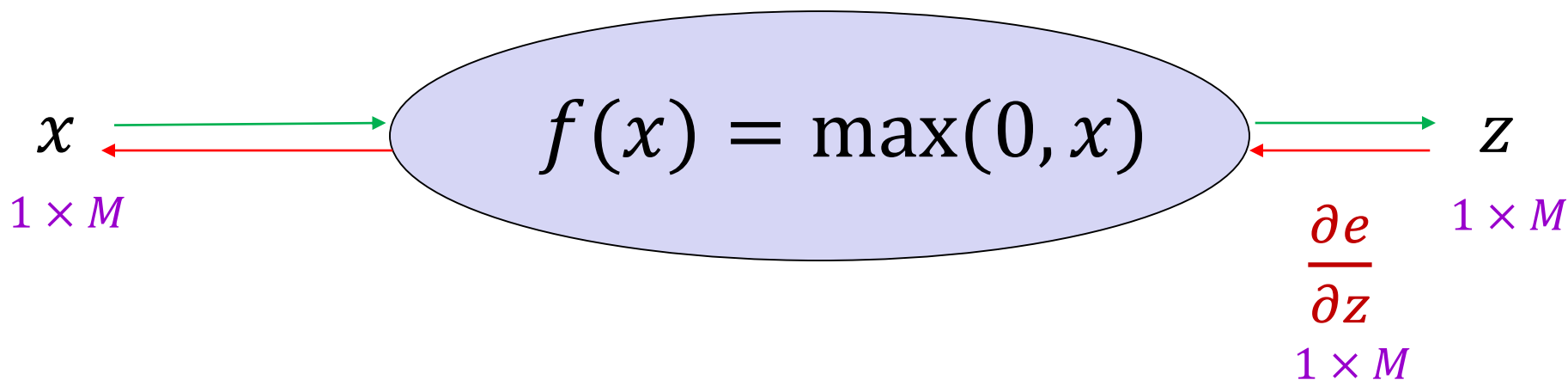


Simple case: Elementwise operation

Simple case: Elementwise operation

$$\frac{\partial z}{\partial x} = \begin{pmatrix} \frac{\partial z_1}{\partial x_1} & \cdots & \frac{\partial z_1}{\partial x_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_M}{\partial x_1} & \cdots & \frac{\partial z_M}{\partial x_M} \end{pmatrix}$$

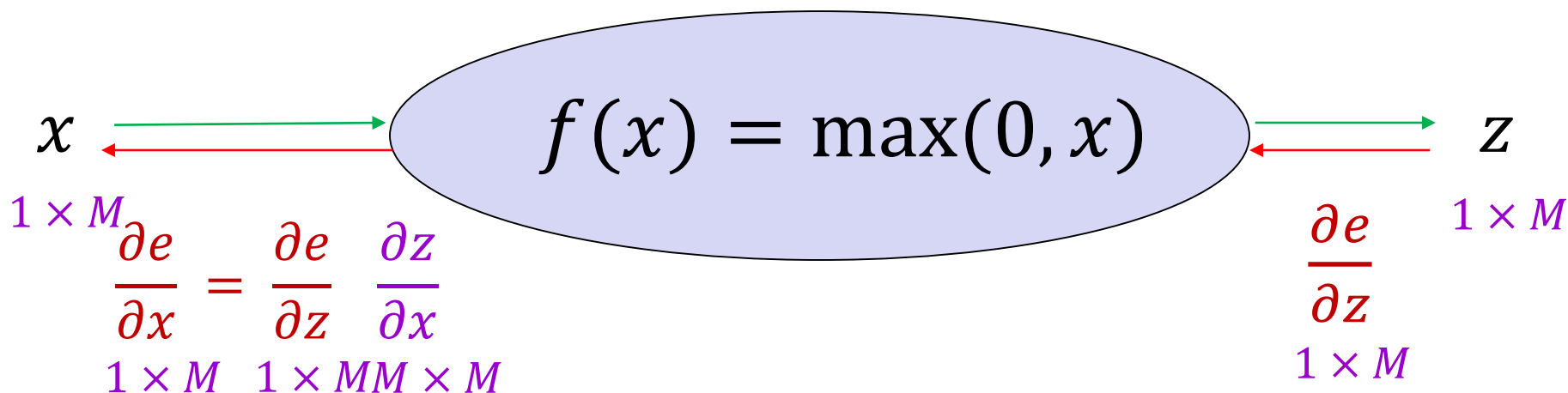
$M \times M$
Jacobian



Simple case: Elementwise operation

$$\frac{\partial z}{\partial x} = \begin{pmatrix} \mathbb{I}[x_1 > 0] & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \mathbb{I}[x_M > 0] \end{pmatrix}$$

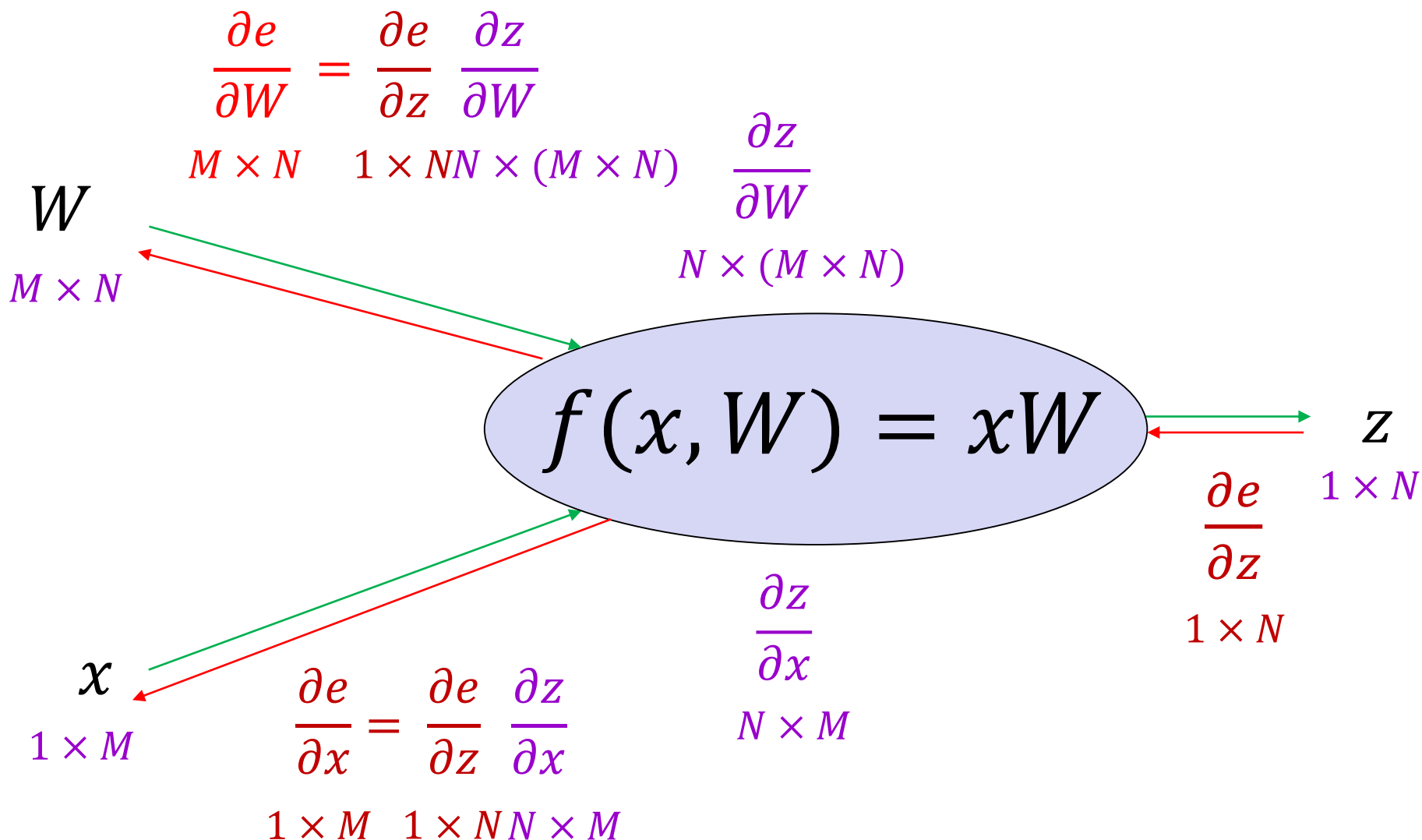
$M \times M$
Jacobian



$$\frac{\partial e}{\partial x_i} = \mathbb{I}[x_i > 0] \frac{\partial e}{\partial z_i}$$

$$\frac{\partial e}{\partial x} = \mathbb{I}[x > 0] * \frac{\partial e}{\partial z}$$

Matrix-vector multiplication



Matrix-vector multiplication

$$(z_1 \quad \dots \quad z_N) = (x_1 \quad \dots \quad x_M) \begin{pmatrix} W_{11} & \dots & W_{1N} \\ \vdots & \ddots & \vdots \\ W_{M1} & \dots & W_{MN} \end{pmatrix} \quad z_j = \sum_{i=1}^M x_i W_{ij}$$

Want: $\frac{\partial e}{\partial x} = \frac{\partial e}{\partial z} \boxed{\frac{\partial z}{\partial x}}$

$1 \times M \quad 1 \times N \quad N \times M$

$$\frac{\partial z_j}{\partial x_i} =$$

j th row, i th column
of Jacobian

$$\frac{\partial z}{\partial x} = W^T$$

$$\frac{\partial e}{\partial x} = \frac{\partial e}{\partial z} \frac{\partial z}{\partial x} = \frac{\partial e}{\partial z} W^T$$

Matrix-vector multiplication

$$(z_1 \quad \dots \quad z_N) = (x_1 \quad \dots \quad x_M) \begin{pmatrix} W_{11} & \dots & W_{1N} \\ \vdots & \ddots & \vdots \\ W_{M1} & \dots & W_{MN} \end{pmatrix}$$

$$z_j = \sum_{i=1}^M x_i W_{ij}$$

Want: $\frac{\partial e}{\partial W}$ $M \times N$ $=$ $\frac{\partial e}{\partial z}$ $\boxed{\frac{\partial z}{\partial W}}$ $1 \times N$ $N \times (M \times N)$

$$\frac{\partial z_k}{\partial W_{ij}} =$$

z_k depends only on
 k th column of W

$$\frac{\partial e}{\partial W_{ij}} =$$

$\kappa = 1$

$$\frac{\partial e}{\partial W} = x^T \frac{\partial e}{\partial z}$$

General tips

- Derive error signal (upstream gradient) directly, avoid explicit computation of huge local derivatives
- Write out expression for a single element of the Jacobian, then deduce the overall formula
- Keep consistent indexing conventions, order of operations
- Use dimension analysis
- **Useful resource:** see Lecture 4 of [Stanford 231n](#) and associated links in the syllabus